CADT IDT

W9 *PRACTICE*

QUIZ APP

Important

- ✓ The **reflection part** will be done in **teams of 2** (*designing*) and 4 (*sharing*)
- ✓ The **coding part** needs to be submitted **individually**

Learning objectives

- ✓ Handle **navigation** between **multiple screens** – *Using a state (not router for now…)*
- ✓ **Pass data** between screens
- ✓ Separate **UI logic** from **business logic**: using a model folder
- ✓ Reflect on the best approaches (***data, states, widgets***) to maintain a clean architecture

How to submit?

- ✓ **Push** your final code on **your GitHub repository**
- ✓ Then **attach the GitHub path** to the MS Team assignment and **turn it in**

Functional Requirements

For this practice (W9)

- ✓ The player can **start the quiz** and **answer each question** one by one
- ✓ Only single choice questions
- ✓ Once finished, the app shows the **score and the questions results**

For Bonus

- ✓ The history of the previous scores can be reviewed
- ✓ The **quiz questions** and **player submission** are persisted in JSON file

For next practice (W10)

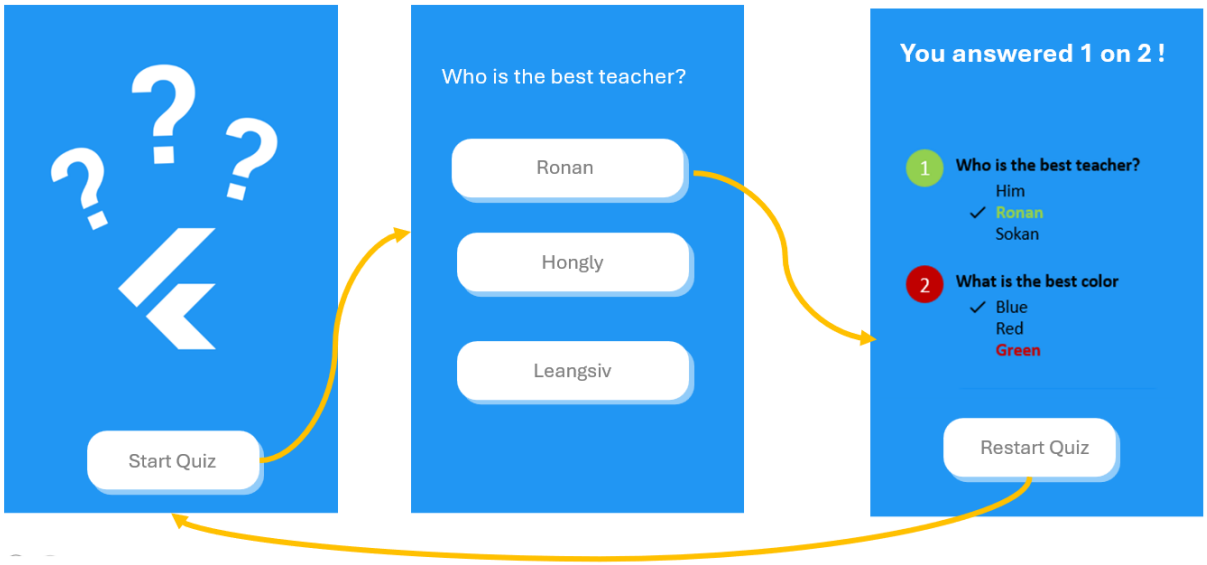
- ✓ The player **enters his/her name** before starting
- ✓ It's possible to **edit the quiz questions**

Non-Functional Requirements

- ✓ The application must **implement the provided user flow and mockups**

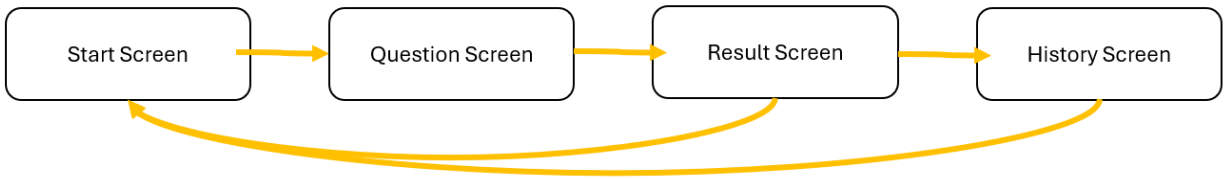
User Flow

For this practice, the following **user flow**/mockup are required:



BONUS

To include the **history of the previous scores**, the **user flow** can evolve as follows:



Layer structure

The application is structured around 3 layers: DATA > DOMAIN > UI

data	Repositories to load **domain objects** from **data sources**
model	Contain the **domain classes**
ui/screen	Screen widgets and sub-screen widgets
ui/widgets	Re usable widgets (button, inputs…)

Here is an **example** of project structure *(just an example, not the correct one)*

```
lib/
 ├─ data/
 │     └─ repositories/
 │           └─ quiz_json_repository.dart
 │            └─ quiz_mock_repository.dart
 ├─ models/
 │     └─ quiz.dart
 │
 └─ ui/
       ├─ screens/
       │     └─ welcome_screen.dart
       │     └─ question_screen.dart
       └─ widgets/
             └─ app_button.dart
              └─ app_button.dart

       main.dart
```

Layer interaction

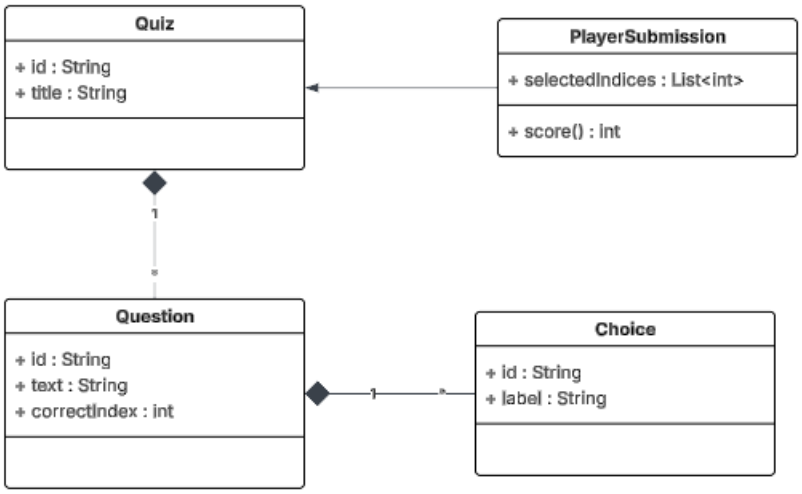
1. The **main** loads the **quiz data** (*from mock data or from a Json file)*
2. The **main** create the **quiz screen**, passing the quiz data as parameter

PART 1 – REFLECTIONS

MODEL

To handle the functional requirements for this practice, and be ready for the next practice, how are you going to structure your model?

Q1 – Drop below the **UML diagram** **of your model**



Q2 – Where do you **keep player submission,** so that you can display the last screen?

- Keep instance of PlayerSubmission inside question_Screen as a StatefulWidget.
- Pass the PlayerSubmission down to child screens via constructors.
- Update it when the user confirms a choice for a question; at the end, use it to compute score and show results.

UI – **Screens**

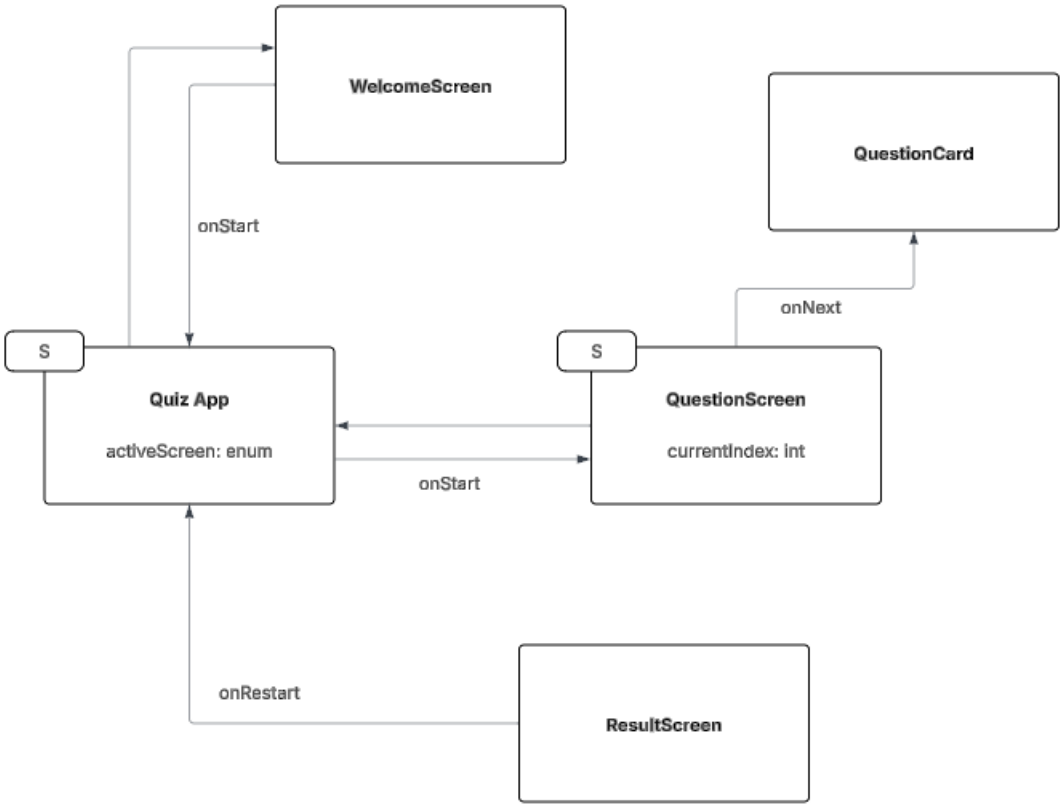
We have 3 screens (start, question and result)

Q3 – Identify for **each widget** their properties

WIDGET	TYPE (SL / SF)	PARAMETERS	STATES
WelcomeScreen	SL	Quiz quiz, VoidCallback onStart	
QuestionScreen	SF	VoidCallback onNext PlayerSubmission submission	Currentindex: int
ResultScreen	SL	VoidCallback onRestart PlayerSubmission submission	

QuizApp	SF		activeScreen: enum

Q4 – Draw the **COMPONENT DIAGRAM** of the application



Q5 – Where and How do you **manage the navigation** to the **next questions** and to the **last result screen**?

Where: In a single stateful parent quiz_app.dart that renders **one screen at a time** based on a simple enum AppStage { start, question, result } and an integer currentIndex.
How:

- On StartScreen.onStart(): set stage = AppStage.question, currentIndex = 0.
- On QuestionScreen.onNext():
  - Append selectedIndex into PlayerSubmission.selectedIndices.
  - If currentIndex < quiz.questions.length - 1 → currentIndex++ (stay in AppStage.question).
  - Else → stage = AppStage.result.
- On ResultScreen.onRestart(): reset submission and go back to AppStage.start.

UI – **Reusable widget**

List down the widget you are **planning to re-use** on different screens (button, card..)

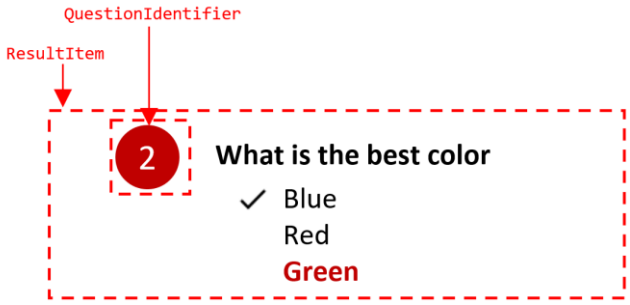
WIDGET	TYPE (SL / SF)	PARAMETERS	STATES
AppButton	SL	• String label • VoidCallback onPressed • bool enabled	
QuestionCard	SL	• Question question, int? selectedIndex • ValueChanged<int> onSelect	
ChoiceTile	SL	• String label • bool selected • VoidCallback onTap	

PART 2 – IMPLEMENTATION

- ✓ The **coding part** needs to be submitted **individually**

HINTS

- ✓ Tip: you can divide each screen into many **stateless screen-widgets**, for example:



This widget takes as parameter a question and a player choice and handle the color computation, the choices highlighting etc..